

企业级项目4 - 基于Qt-Linux内核的微桌面管理系统

4.1 Qt5 中英文软键盘 (im) 部署

1. 项目介绍

本项目是一个 Qt5 平台输入法插件，支持中英文输入，适用于嵌入式 Linux 设备的触摸屏输入场景。

- 工程结构

```
im/
├─ im.pro           # 主工程文件
├─ common.pri       # 公共配置
├─ core/            # 核心功能库 (libcore.so)
├─ ui/              # UI 组件库 (libui.so)
└─ imvirtualkeyboard/ # 输入法插件 (libIM.so)
```

- 输出文件

编译后生成以下文件：

文件	说明
lib/libcore.so	核心功能库
lib/libui.so	UI 组件库
plugins/platforminputcontexts/libIM.so	输入法插件
pinyininputmethod/pinyin/data/dict_pinyin.dat	拼音词典

2. Ubuntu 22.04 部署

- 部署文件

重要：若你的 Qt 应用（如 Wifi）是用**本地 Qt**（如 /home/linux/Qt/5.15.2/gcc_64）编译的，插件必须放到**该 Qt 的插件目录**或通过 QT_PLUGIN_PATH 指定，不能只拷贝到 /usr/lib/qt5/（系统 Qt5 路径）。

```
# 输出目录
OUTPUT_DIR="/home/linux/emb/14-desktop/desktop/im"

# 1. 拷贝动态库到系统目录（或应用程序目录）
sudo cp $OUTPUT_DIR/lib/libcore.so* /usr/lib/
sudo cp $OUTPUT_DIR/lib/libui.so* /usr/lib/

# 2. 拷贝输入法插件（二选一）
# 方式 A：应用使用【系统 Qt5】时，拷贝到系统路径
sudo mkdir -p /usr/lib/qt5/plugins/platforminputcontexts
sudo cp $OUTPUT_DIR/plugins/platforminputcontexts/libIM.so \
    /usr/lib/qt5/plugins/platforminputcontexts/

# 方式 B：应用使用【本地 Qt 5.15.2】时，拷贝到该 Qt 的插件目录
# 用 ldd your_app 查看应用链接的 Qt 路径，再拷贝到对应 plugins 目录
sudo cp $OUTPUT_DIR/plugins/platforminputcontexts/libIM.so \
    /home/linux/Qt/5.15.2/gcc_64/plugins/platforminputcontexts/

# 3. 拷贝拼音词典
sudo mkdir -p /usr/share/pinyin/3rdparty/pinyin/data
sudo cp /home/linux/emb/14-desktop/im/imvirtualkeyboard/pinyininputmethod/pinyin/data/dict_pinyin.dat \
    /usr/share/pinyin/3rdparty/pinyin/data/

# 4. 更新动态库缓存
sudo ldconfig
```

不拷贝系统目录时：启动应用前设置 QT_PLUGIN_PATH=\$OUTPUT_DIR/plugins，Qt 会从此路径加载插件。

- 配置环境变量

```
# 在 ~/.bashrc 或 /etc/profile 中添加
export QT_IM_MODULE=im
```

或者在启动 Qt 应用前设置：

```
QT_IM_MODULE=im ./your_qt_app
```

- 测试

```
# 启动任意 Qt 应用测试
QT_IM_MODULE=im qtcreeator
```

点击输入框，软键盘应自动弹出。

3. i.MX6ULL 部署

- 交叉编译环境准备

```
# 设备 IP 地址
DEVICE_IP=192.168.2.7

# 1. 拷贝动态库
scp ../desktop/im/lib/*.so root@$DEVICE_IP:/usr/lib/

# 2. 拷贝输入法插件
ssh root@$DEVICE_IP "mkdir -p /usr/lib/qt/plugins/platforminputcontexts"
scp ../desktop/im/plugins/platforminputcontexts/libIM.so \
    root@$DEVICE_IP:/usr/lib/qt/plugins/platforminputcontexts/

# 3. 拷贝拼音词典
ssh root@$DEVICE_IP "mkdir -p /usr/share/pinyin/3rdparty/pinyin/data"
scp imvirtualkeyboard/pinyininputmethod/pinyin/data/dict_pinyin.dat \
    root@$DEVICE_IP:/usr/share/pinyin/3rdparty/pinyin/data/
```

- 配置环境变量

在设备上创建或编辑 /etc/profile.d/qt-im.sh：

```
#!/bin/sh
export QT_IM_MODULE=im
export QT_PLUGIN_PATH=/usr/lib/qt/plugins
```

或者直接在 /etc/profile 中添加：

```
export QT_IM_MODULE=im
```

- 重启生效

```
# 重启设备
reboot

# 或者重新加载环境变量
source /etc/profile
```

- 测试验证

```
# 在设备上运行 Qt 应用
cd /root/desktop/wifi
./wifi

# 点击输入框，软键盘应自动弹出
```

- 显示模式

模式	说明	设置方法
跟随模式	键盘跟随输入框位置显示	默认
底部固定	键盘固定在窗口底部	qApp->setProperty("bottom", true);

- 代码示例

```
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication app(argc, argv);

    // 可选：设置键盘固定在底部显示
    // app.setProperty("bottom", true);

    // ... 创建窗口和输入框

    return app.exec();
}
```

- 文件清单

文件	设备路径	说明
libcore.so	/usr/lib/	核心库
libui.so	/usr/lib/	UI 库
libIM.so	/usr/lib/qt/plugins/platforminputcontexts/	输入法插件
dict_pinyin.dat	/usr/share/pinyin/3rdparty/pinyin/data/	拼音词典

注意：不同 Qt 版本的插件路径可能不同，请根据实际情况调整：

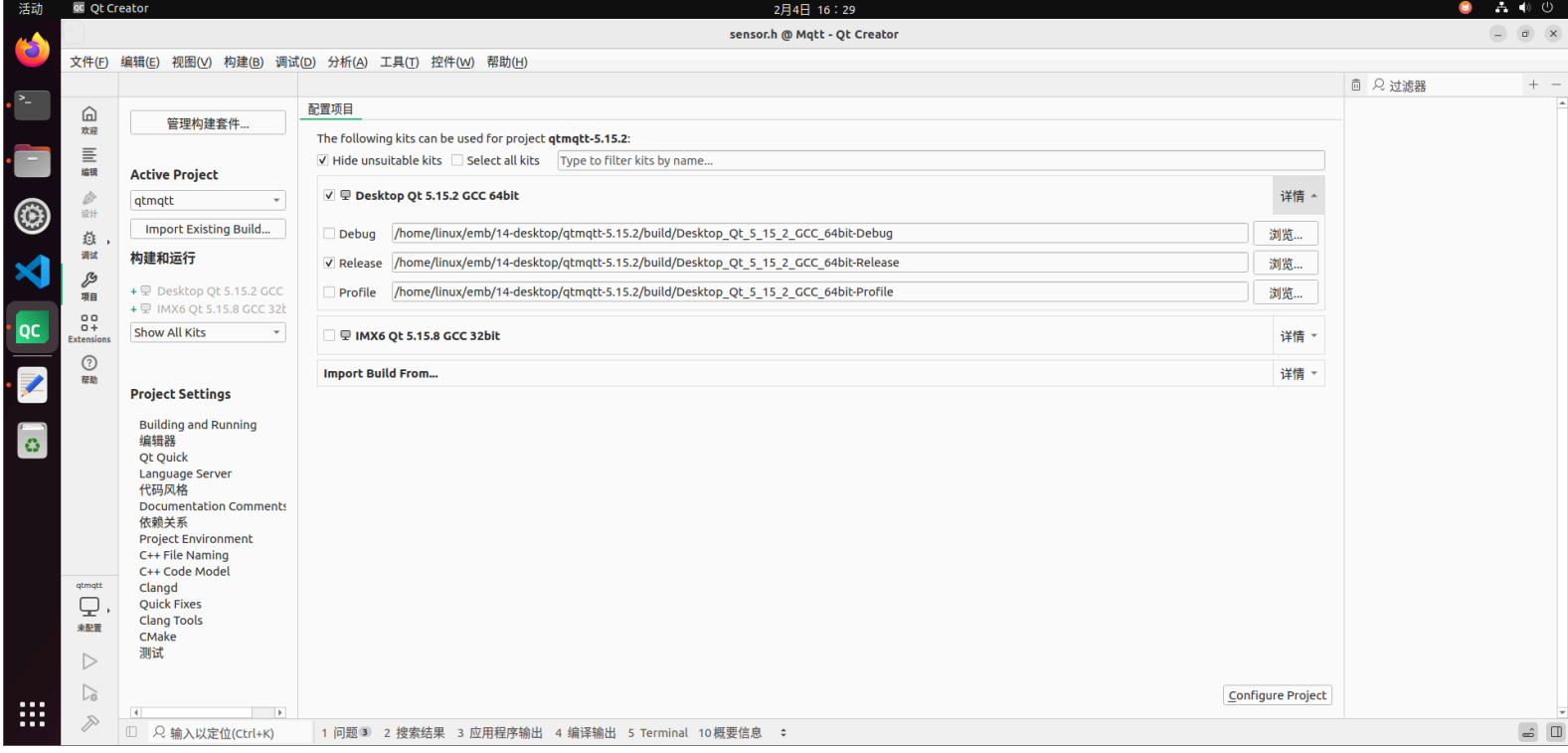
- Qt 5.15 (Buildroot): `/usr/lib/qt/plugins/platforminputcontexts/`
- Qt 5.15 (Ubuntu): `/usr/lib/qt5/plugins/platforminputcontexts/`

4.2 Ubuntu22.04编译QMqtt库

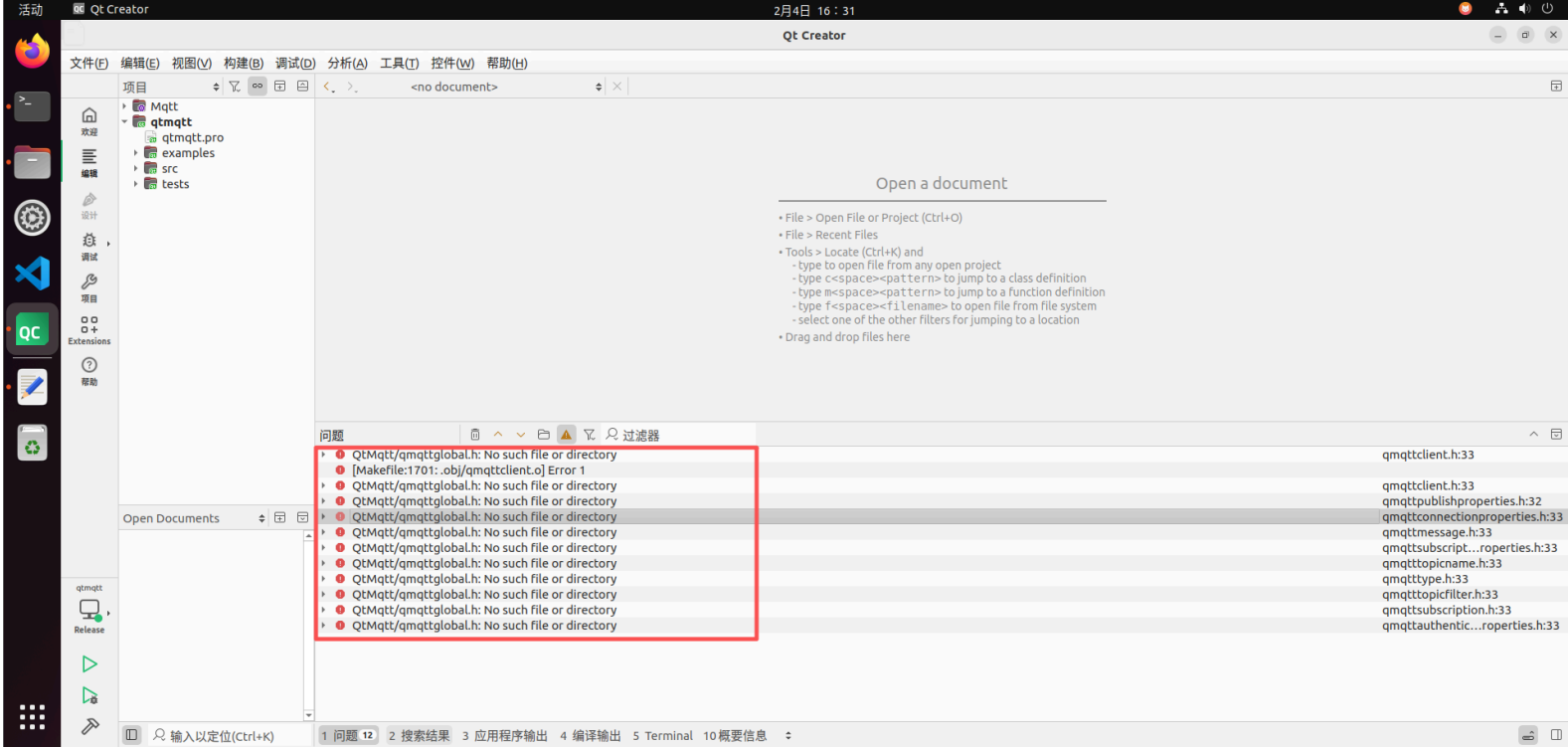
1. Ubuntu下编译Mqtt库

<https://github.com/qt/qtmqtt/tree/5.15.2>

打开QT工程，使用Desktop开发套件。



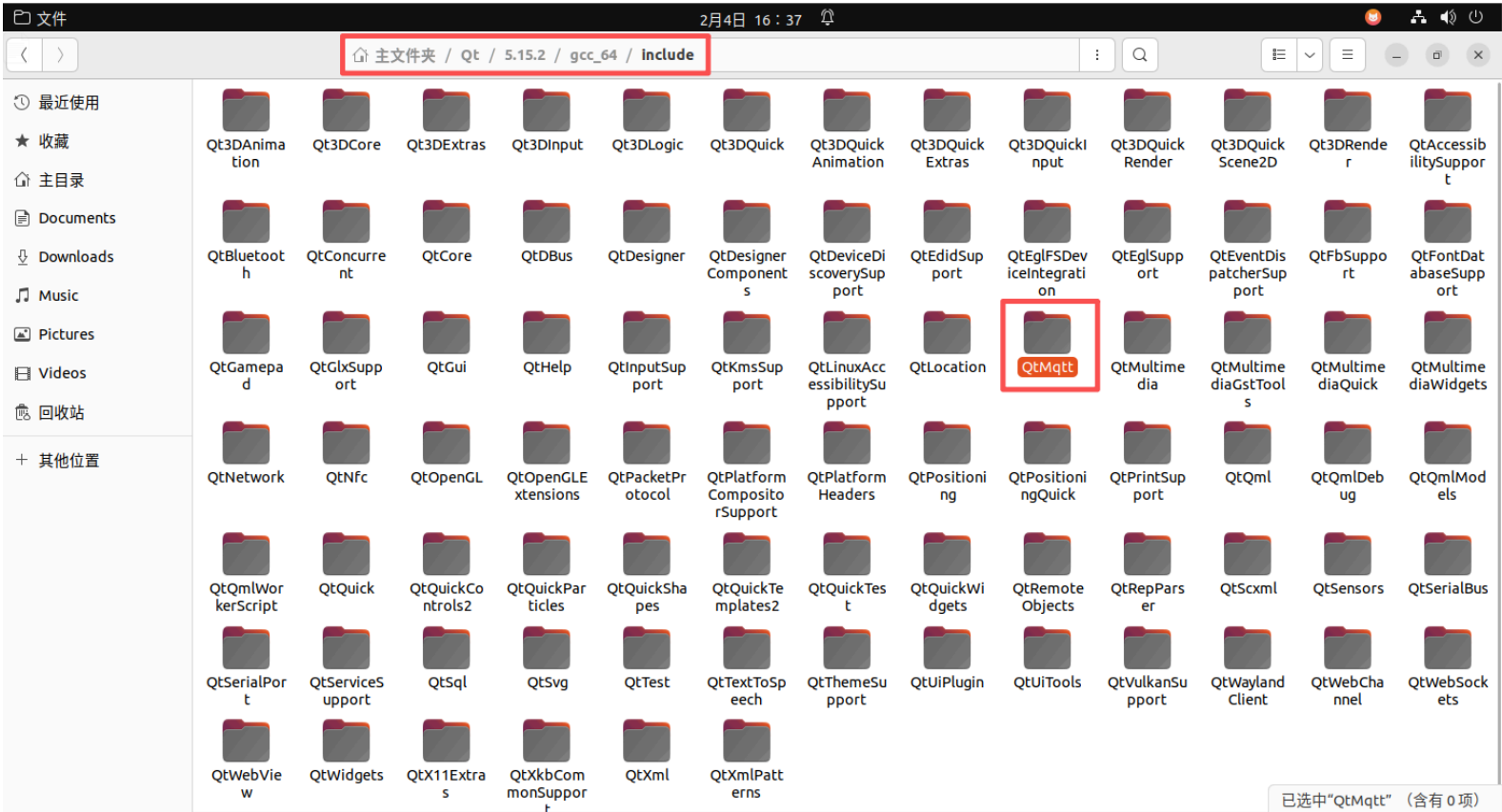
编译QT工程，会出现错误，如下：



解决办法，QT安装路径的对应include路径下新建一个文件夹QtMqtt，目录如下：

`/home/linux/Qt/5.15.2/gcc_64/include`

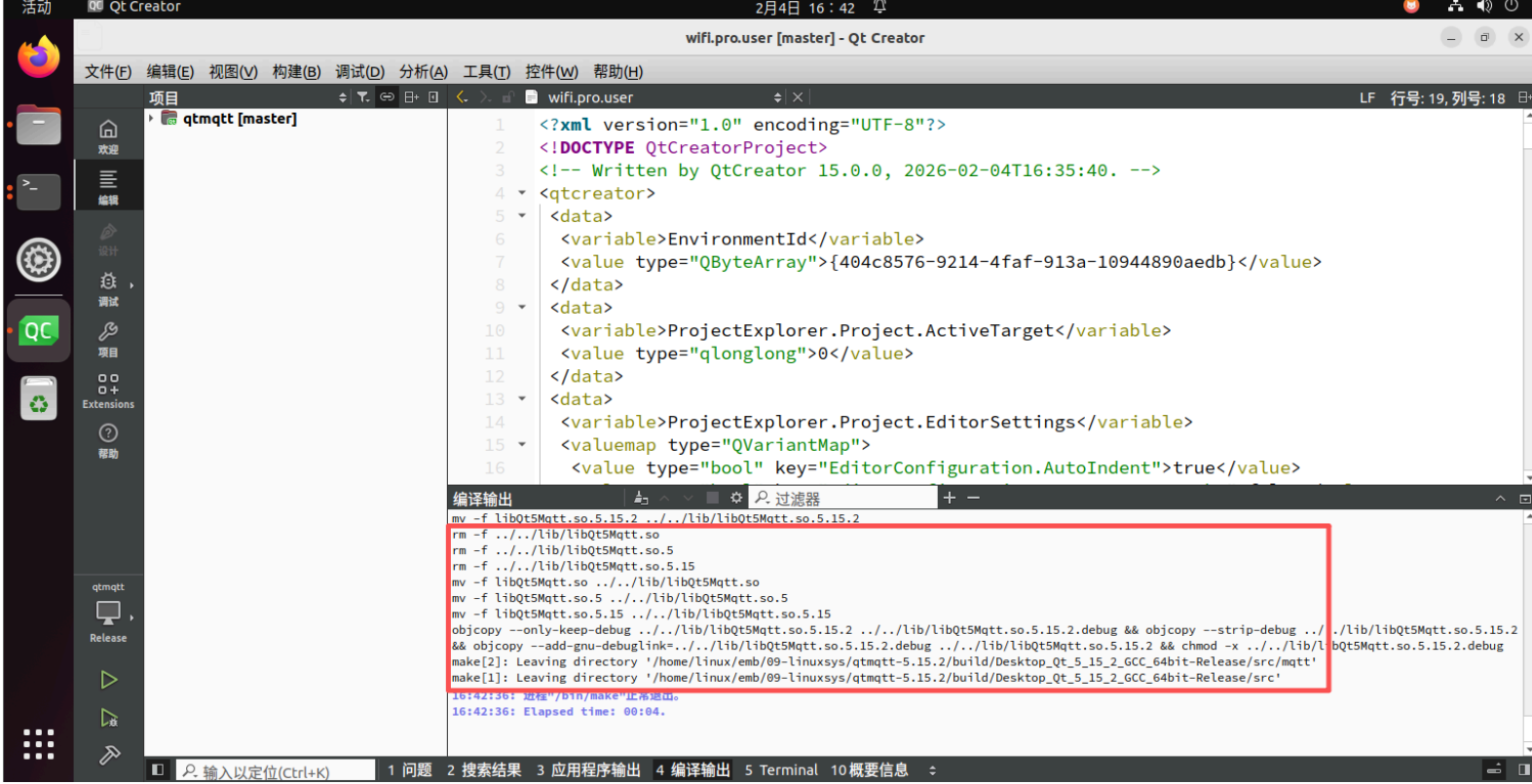
根据自己的目录进行创建



复制 qtmqtt\qt5mqtt-5.15.2\src\mqtt 下的 *.h 文件到 QtMqtt 目录下。



这时候再次编译（Release模式）QtMqtt源码，编译完成后得到以下文件。



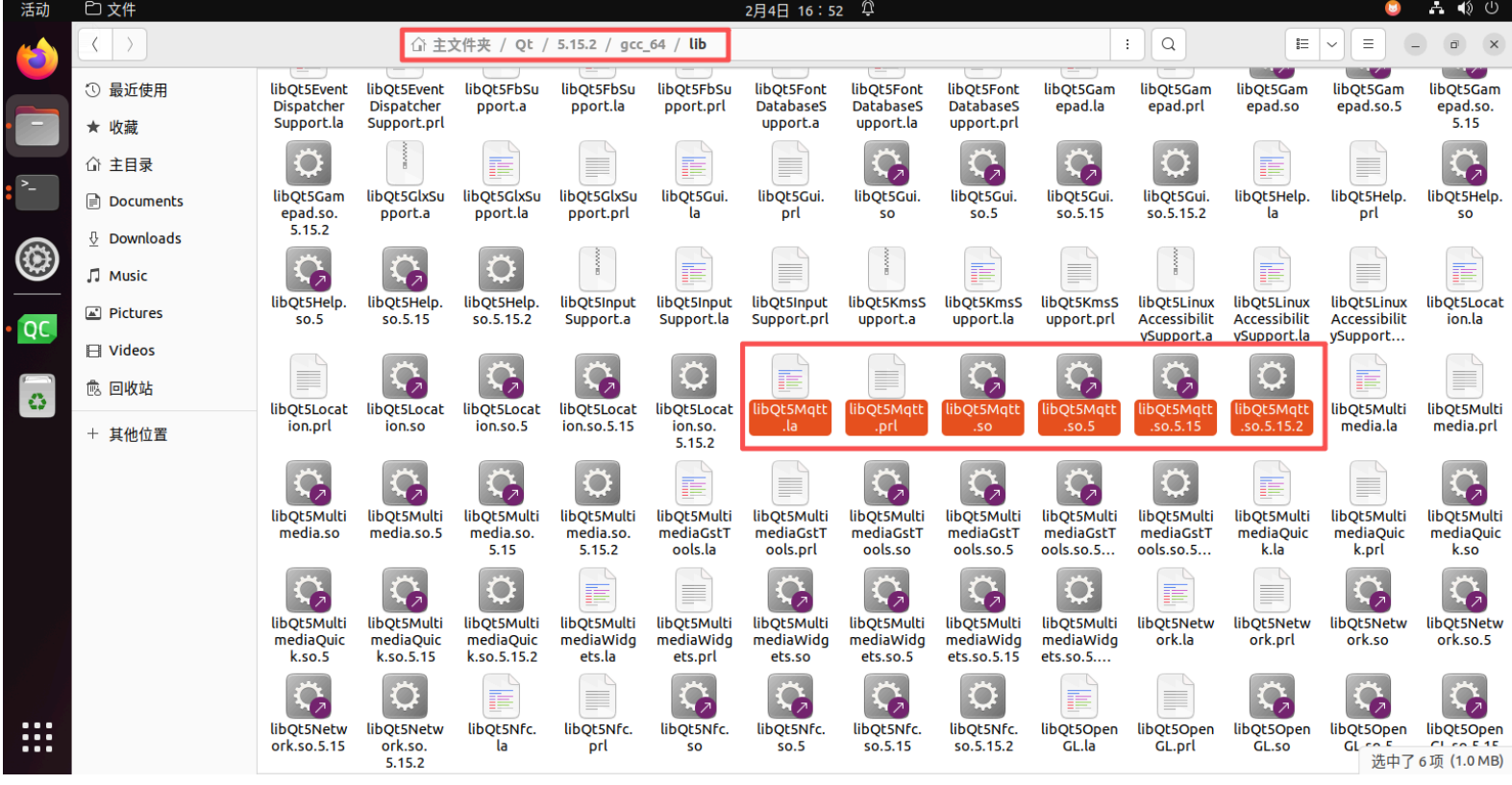
lib目录下就是我们需要的库文件：



复制一下几个文件到qt安装目录下

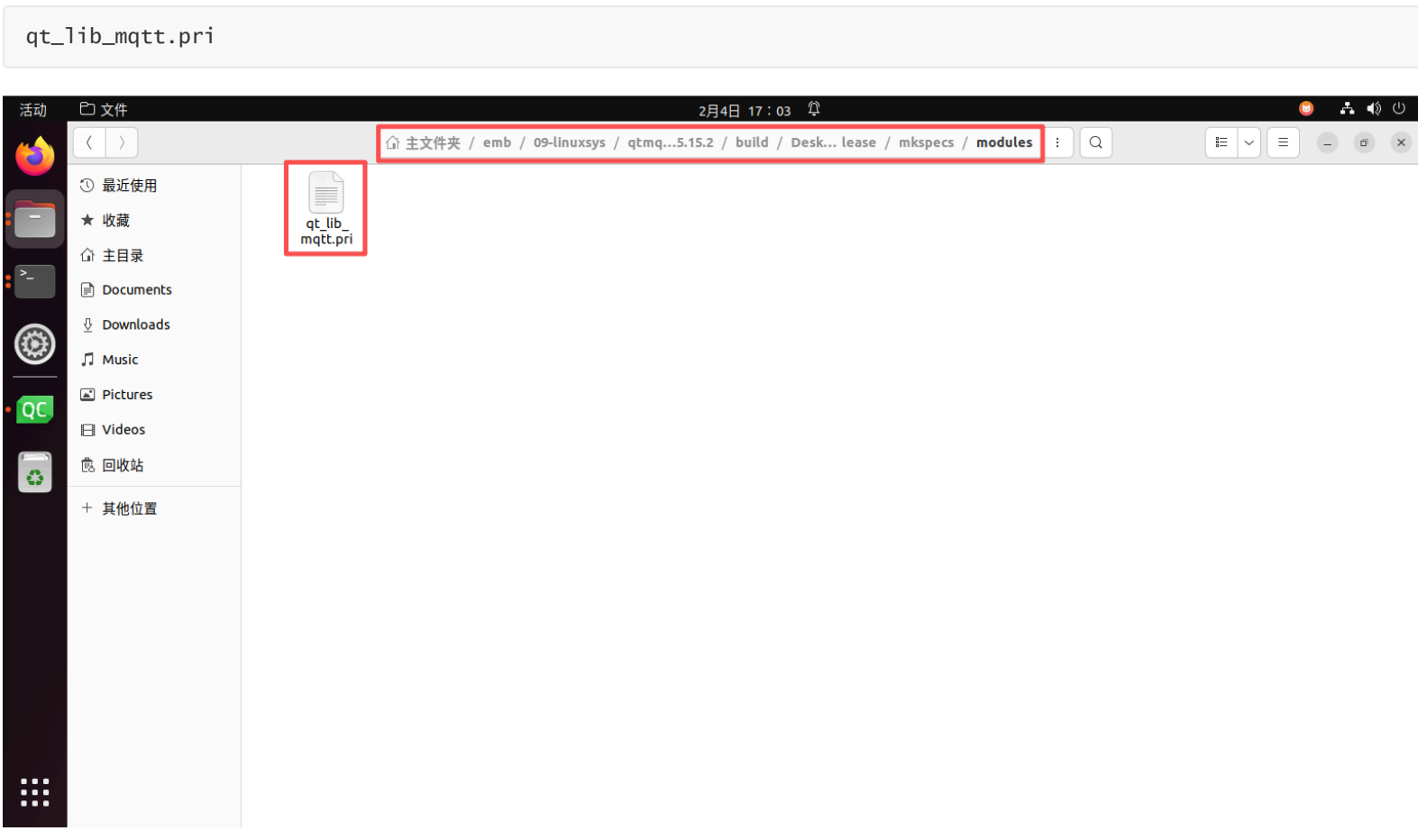
```
├─ libQt5Mqtt.la
├─ libQt5Mqtt.prl
├─ libQt5Mqtt.so -> libQt5Mqtt.so.5.15.2
├─ libQt5Mqtt.so.5 -> libQt5Mqtt.so.5.15.2
├─ libQt5Mqtt.so.5.15 -> libQt5Mqtt.so.5.15.2
├─ libQt5Mqtt.so.5.15.2
```

复制并粘贴到Qt安装目录下的lib文件夹中：

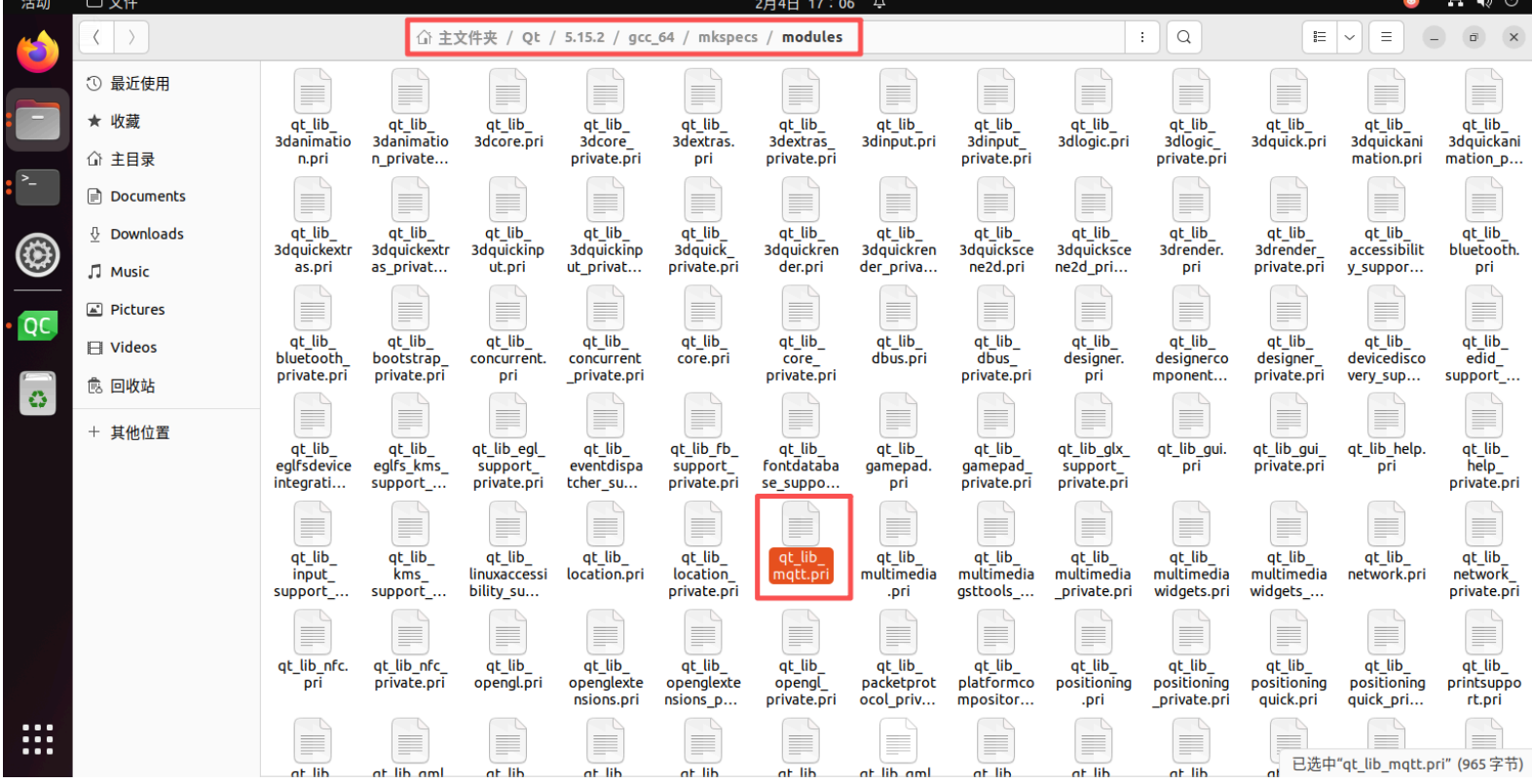


- 配置模块

复制模块文件 qt_lib_mqtt.pri



复制到qt安装目录下



配置完毕，这种方式配置在新建工程中引用只需要引入模块就可以直接使用了

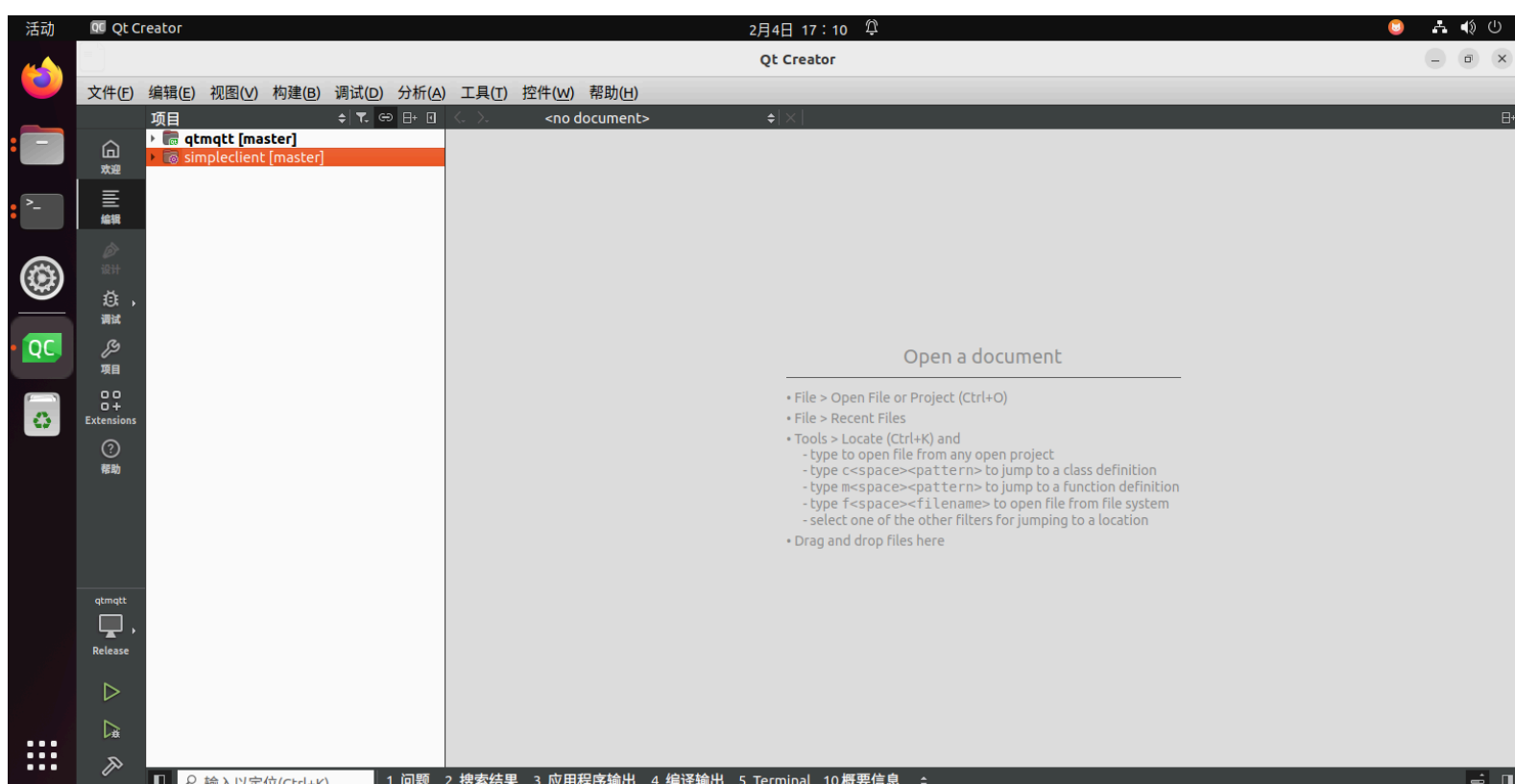
```
QT += mqtt
```

包含头文件

```
#include <QtMqtt/QtMqtt>
```

3. 编译并运行simpleclient工程

复制 qt5mqtt-5.15.2\examples\mqtt\simpleclient 工程到 qt5mqtt-5.15.2 的同级目录下，并打开工程

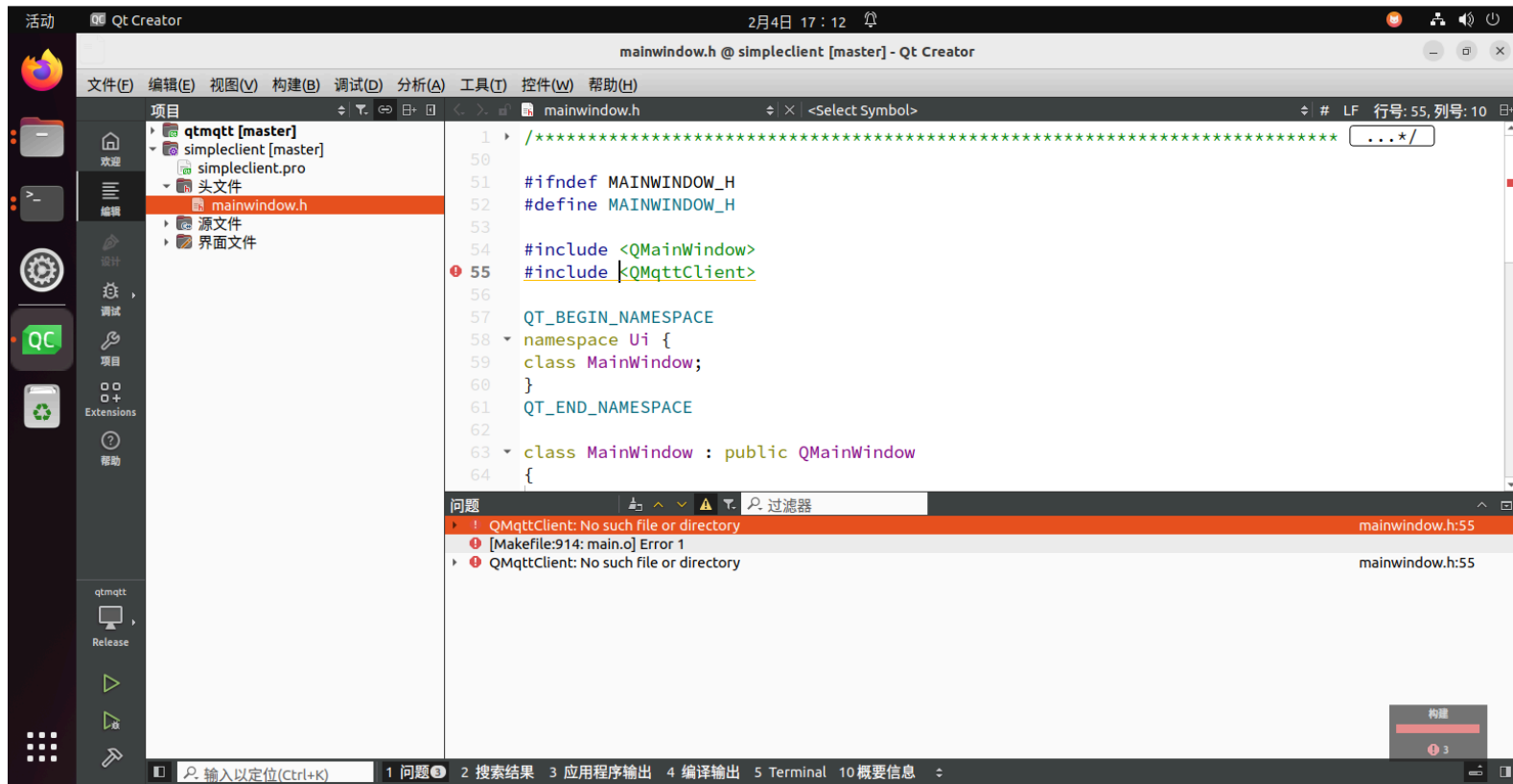


将该示例打开，有个地方需要改一下，打开Pro文件

```
20 SOURCES += main.cpp\  
21 |      |      mainwindow.cpp  
22  
23 HEADERS += mainwindow.h  
24 FORMS    += mainwindow.ui  
25  
26 # target.path = $$[QT_INSTALL_EXAMPLES]/mqtt/simpleclient  
27 # INSTALLS += target  
28 |
```

将以上两行注释掉，要不然会有依赖，无法独立运行。

- 编译 (Release模式) 工程。



解决：

无法打开包括文件: "QMqttClient": No such file or directory

- 更换头文件

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QQtClient> // 更换这个头文件

QT_BEGIN_NAMESPACE
namespace Ui {
class MainWindow;
}
QT_END_NAMESPACE
```

修改为：

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

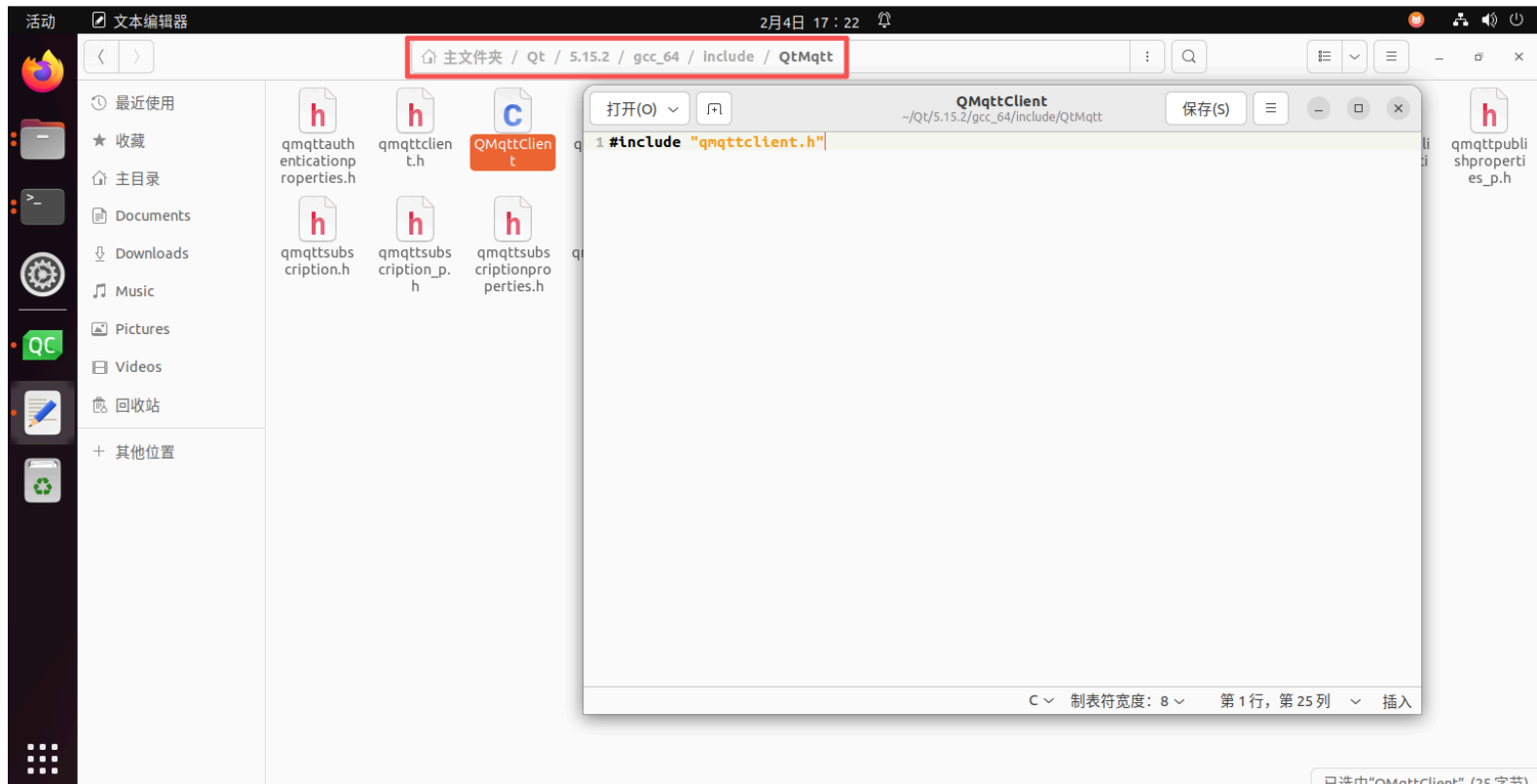
#include <QMainWindow>
// #include <QQtClient>
#include <QtMqtt/QtMqttClient>

QT_BEGIN_NAMESPACE
namespace Ui {
class MainWindow;
}

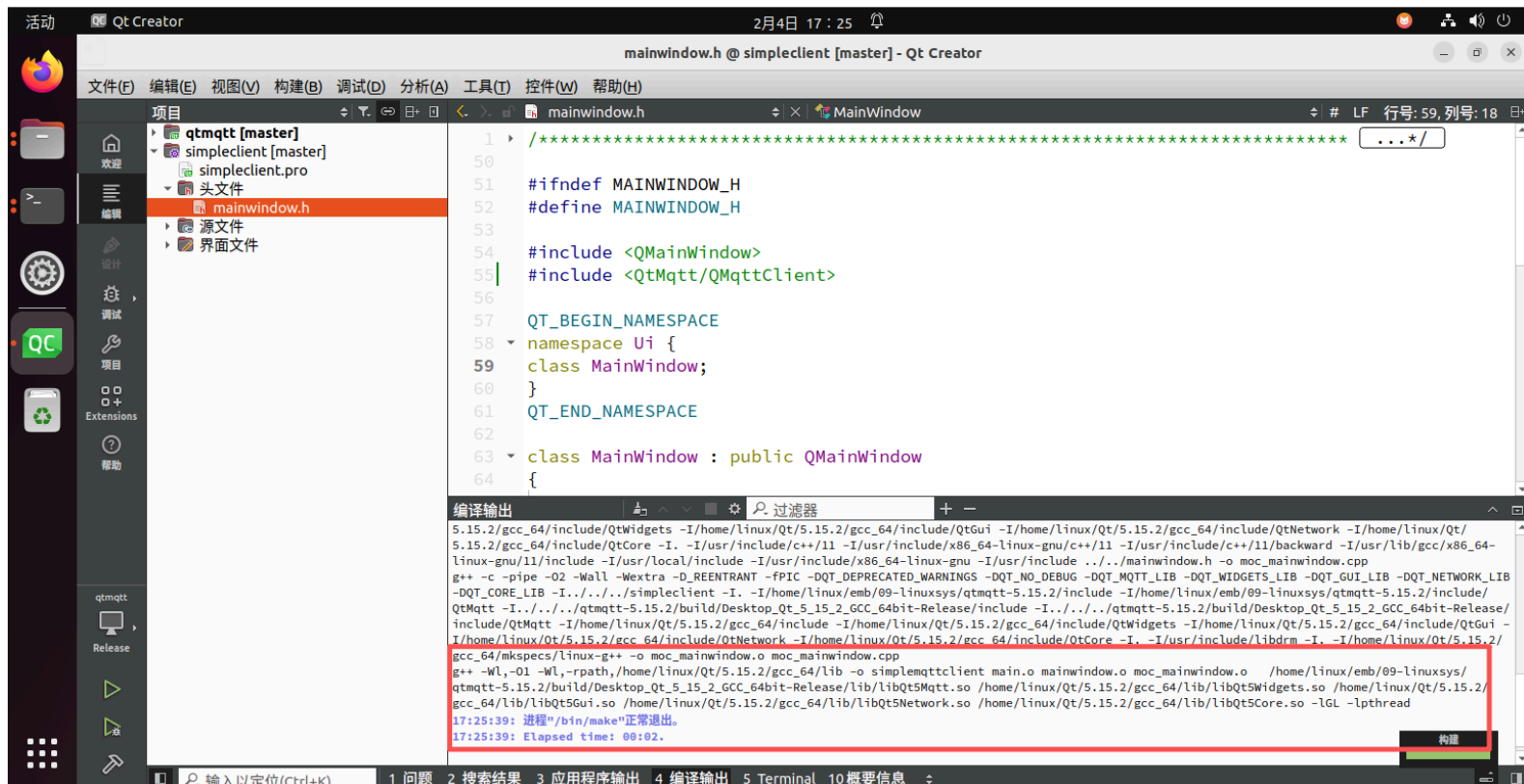
```

在 /home/linux/Qt/5.15.2/gcc_64/include/QtMqtt 创建 QMqttClient 文件，文件内部包含头文件

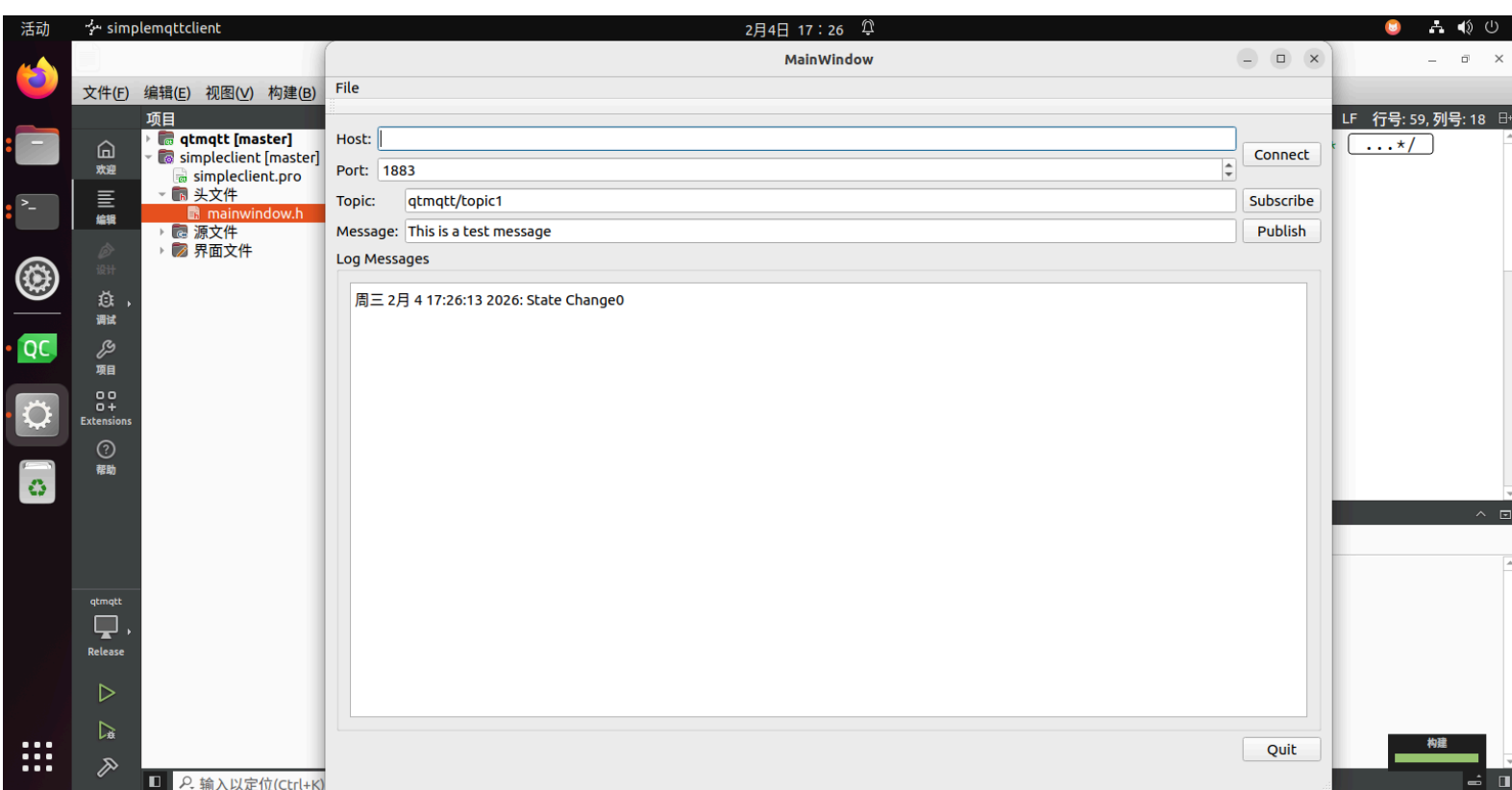
```
#include "qmqttclient.h"
```



再次编译 (Release模式) 工程, 编译通过。



执行可执行程序



4.3 Buildroot编译Mqtt库

1. buildroot添加QtMqtt库

配置buildroot对QtMqtt的支持

```
Target packages  --->
  Graphic libraries and applications (graphic/text)  --->
    [*] Qt5  --->
      [*] qt5mqtt
```

编译文件系统

```
>>> Generating filesystem image rootfs.tar
mkdir -p /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/images
rm -rf /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/build/buildroot-fs/tar
mkdir -p /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/build/buildroot-fs/tar
rsync -auH --exclude=/THIS_IS_NOT_YOUR_ROOT_FILESYSTEM /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/target/ /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/build/buildroot-fs/tar/target
echo '#!/bin/sh' > /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/build/buildroot-fs/tar/fakeroot
echo "set -e" >> /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/build/buildroot-fs/tar/fakeroot
echo "chown -h -R 0:0 /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/build/buildroot-fs/tar/target" >> /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/build/buildroot-fs/tar/fakeroot
PATH="/home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/host/bin:/home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/host/sbin:/home/linux/per15/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin" /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/support/scripts/mkusers
/home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/build/buildroot-fs/full_users_table.txt
/home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/build/buildroot-fs/tar/target >> /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/build/buildroot-fs/tar/fakeroot
echo "/home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/host/bin/makedevs -d /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/build/buildroot-fs/full_devices_table.txt /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/build/buildroot-fs/tar/target" >> /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/build/buildroot-fs/tar/fakeroot
printf '    rm -rf /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/build/buildroot-fs/tar/target//etc/udev/hwdb.d/\n' >> /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/build/buildroot-fs/tar/fakeroot
printf '    \n' >> /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/build/buildroot-fs/tar/fakeroot
printf '    \n' >> /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/build/buildroot-fs/tar/fakeroot
printf '    (cd /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/build/buildroot-fs/tar/target; find -print0 | LC_ALL=C sort -z | tar --pax-option=exthdr.name=%d/PaxHeaders/%f,atime:=0,ctime:=0 -cf /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/images/rootfs.tar --null --xattrs-include=\'\''\'\'\' --no-recursion -T - --numeric-owner)\n' >> /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/build/buildroot-fs/tar/fakeroot
chmod a+x /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/build/buildroot-fs/tar/fakeroot
PATH="/home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/host/bin:/home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/host/sbin:/home/linux/per15/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin" FAKEROOTDONTTRYCHOWN=1 /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/host/bin/fakeroot
-- /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/build/buildroot-fs/tar/fakeroot
rootdir=/home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/build/buildroot-fs/tar/target
table='/home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/build/buildroot-fs/full_devices_table.txt'
PATH="/home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/host/bin:/home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/host/sbin:/home/linux/per15/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin" bzip2 -9 -c /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/images/rootfs.tar
> /home/linux/emb2307/12-linuxsys/buildroot-2021.02.12/output/images/rootfs.tar.bz2
linux@ubuntu:~/emb2307/12-linuxsys/buildroot-2021.02.12$
```

编译成功后，制作的文件系统默认已经安装了Mqtt库，imx6平台可以运行mqtt的程序。

```
linux@ubuntu:~/emb2307/12-linuxsys/buildroot-2021.02.12$ ls output/target/usr/lib/libQt5Mqtt.so* -l
lrwxrwxrwx 1 linux linux    20 1月   9 17:11 output/target/usr/lib/libQt5Mqtt.so -> libQt5Mqtt.so.5.15.2
lrwxrwxrwx 1 linux linux    20 1月   9 17:11 output/target/usr/lib/libQt5Mqtt.so.5 -> libQt5Mqtt.so.5.15.2
lrwxrwxrwx 1 linux linux    20 1月   9 17:11 output/target/usr/lib/libQt5Mqtt.so.5.15 -> libQt5Mqtt.so.5.15.2
-rwxr-xr-x 1 linux linux 133852 1月   9 18:45 output/target/usr/lib/libQt5Mqtt.so.5.15.2
linux@ubuntu:~/emb2307/12-linuxsys/buildroot-2021.02.12$
```

4.4 设计自己的通信协议

1. 控制设备的json字符串

控制设备开

```
{
  "LED1":true,
  "LED2":true,
  "BEEP":true
}
```

控制设备关

```
{
  "LED1":false,
  "LED2":false,
  "BEEP":false
}
```

2. 传感器数据上传的json字符串

```
{
  "TP":25.5,    // °C
  "RH":45.5,    // %
  "CU":201,     // ma
  "VL":4.85,    // V
  "PW":809,     // mW
  "VR":3.0,     // V
  "CPU":33.2    // °C
}

// TP : 温度      类型为 实数
// RH : 湿度      类型为 实数
// CU : 电流      类型为 整数
// VL : 电压      类型为 实数
// PW : 功率      类型为 整数
// VR : 电位器电压 类型为 实数
// CPU : CPU温度   类型为 实数
```

3. 主题设置

IM6ULL 的发送或上行主题为

```
IMX6ULL/UPLoad/cpuid
// 例如
IMX6ULL/UPLoad/223141d768e08805
```

IMX6ULL 的接收或下行送主题为

```
IMX6ULL/DownLoad/cpuid
// 例如
IMX6ULL/DownLoad/223141d768e08805
```

设备全部工作

```
{
  "LED1":true,
  "LED2":true,
  "IO1":true,
  "BEEP":true
}
```

设备全部不工作

```
{
  "LED1":false,
  "LED2":false,
  "IO1":false,
  "BEEP":false
}
```